

T3: Минимальный прототип Visual-first AI Website Generator

Цель

Собрать минимальный локальный прототип, который доказывает главный пайплайн:

бриф пользователя → картинка hero-блока → описание дизайна → frontend-код → скриншот результата → сравнение с референсом → отчёт по отличиям

Это не SaaS, не личный кабинет и не полноценный конструктор сайтов.

Главная цель — проверить, может ли система стабильно превращать утверждённый визуальный референс в похожий frontend-код.

1. Что нужно сделать в первой версии

Нужно реализовать только один сценарий:

1. Взять текстовый бриф пользователя.
 2. Сгенерировать по нему визуальный референс одного hero-блока.
 3. Проанализировать картинку и создать описание дизайна.
 4. Сгенерировать frontend-код по картинке и описанию.
 5. Запустить проект локально.
 6. Сделать скриншот результата.
 7. Сравнить исходную картинку и скриншот.
 8. Создать отчёт с визуальными отличиями.
-

2. Что НЕ нужно делать

В этой версии не нужно делать:

- регистрацию;
 - авторизацию;
 - оплату;
 - личный кабинет;
 - историю проектов;
 - drag-and-drop редактор;
 - Figma export;
 - deploy на Vercel;
 - генерацию полного сайта;
 - многостраничные сайты;
-

- CMS;
- формы;
- backend;
- адаптивную кастомизацию руками;
- красивый пользовательский интерфейс.

Первый прототип может работать через локальный запуск команды.

3. Формат входных данных

На входе должен быть файл:

```
input/brief.md
```

Пример содержимого:

Сделай первый экран лендинга для AI-сервиса финансовой аналитики.

Стиль:

- тёмный
- дорогой
- современный
- крупная типографика
- фиолетово-синие акценты
- карточка продукта справа

Текст:

Заголовок: AI-аналитика для финансовых команд

Описание: Получайте инсайты, прогнозы и отчёты быстрее без ручной рутины.

Основная кнопка: Начать бесплатно

Вторая кнопка: Смотреть демо

4. Структура проекта

Нужно создать такую структуру:

```
project/  
  input/  
    brief.md  
  
  runs/  
    run-001/  
      reference/  
        hero-reference.png  
  
    design/  
      design-description.md  
      design-tokens.json  
  
  code/
```

```
frontend-project/  
  
screenshots/  
  rendered-desktop.png  
  rendered-mobile.png  
  
qa/  
  visual-report.md  
  diff.png  
  
status.json
```

Каждый запуск должен сохраняться в отдельную папку внутри runs/.

5. Шаг 1 – подготовка брифа

Нужно взять сырой текст пользователя из input/brief.md и превратить его во внутреннее структурированное описание.

На выходе нужно получить:

runs/run-001/project-spec.json

Внутри должны быть:

- тип сайта;
- тип блока;
- стиль;
- аудитория;
- обязательные элементы;
- текст заголовка;
- текст описания;
- текст кнопок;
- визуальные пожелания.

Пример:

```
{  
  "siteType": "landing",  
  "sectionType": "hero",  
  "style": ["dark", "premium", "modern", "saas"],  
  "audience": "finance teams",  
  "requiredElements": [  
    "headline",  
    "description",  
    "primaryButton",  
    "secondaryButton",  
    "productCard"  
  ],  
  "copy": {  
    "headline": "AI-аналитика для финансовых команд",  
    "description": "Получайте инсайты, прогнозы и отчёты быстрее без ручной  
рутины.",
```

```
    "primaryButton": "Начать бесплатно",  
    "secondaryButton": "Смотреть демо"  
  }  
}
```

6. Шаг 2 – генерация визуального референса

Нужно сгенерировать изображение hero-блока.

Требования к изображению:

- только один hero-блок;
- без браузерной рамки;
- без лишних секций;
- размер примерно 1440x900;
- дизайн должен быть реализуемым через frontend;
- не использовать слишком сложные 3D-объекты;
- не использовать невозможные декоративные элементы;
- текст должен быть читаемым;
- структура должна быть понятной.

На выходе:

runs/run-001/reference/hero-reference.png

7. Шаг 3 – проверка визуала

После генерации картинки нужно проверить, подходит ли она для дальнейшей вёрстки.

Проверить:

- есть ли понятная сетка;
- читается ли текст;
- понятно ли расположение элементов;
- нет ли слишком сложной графики;
- можно ли повторить дизайн через CSS/Tailwind;
- нет ли лишних блоков;
- соответствует ли картинка брифу.

Если картинка плохая, нужно создать отчёт:

runs/run-001/qa/reference-validation.md

И пометить запуск как:

```
{  
  "status": "reference_failed"  
}
```

Если картинка подходит, продолжить.

8. Шаг 4 – описание дизайна

Нужно проанализировать картинку и создать файл:

runs/run-001/design/design-description.md

В описании должны быть:

Фон

основной цвет;

градиенты;

декоративные элементы.

Сетка

сколько колонок;

где расположен текст;

где расположена карточка;

примерные отступы.

Типографика

размер заголовка;

жирность;

высота строки;

цвет текста;

цвет вторичного текста.

Кнопки

внешний вид основной кнопки;

внешний вид вторичной кнопки;

скругление;

размеры;

цвета.

Карточки

форма;

фон;

обводка;

тень;

внутренние элементы.

Адаптив

как блок должен выглядеть на мобильном экране.

9. Шаг 5 – дизайн-токены

Нужно создать файл:

runs/run-001/design/design-tokens.json

Примерная структура:

```
{
  "colors": {
    "background": "#050816",
    "textPrimary": "#FFFFFF",
    "textSecondary": "#A7B0C0",
    "accent": "#7C3AED",
    "surface": "rgba(255,255,255,0.06)"
  },
  "layout": {
    "containerWidth": "1200px",
    "sectionPaddingY": "96px",
```

```

    "sectionPaddingX": "32px",
    "columns": 2
  },
  "typography": {
    "headlineSize": "72px",
    "headlineWeight": 700,
    "bodySize": "18px"
  },
  "components": {
    "buttonRadius": "999px",
    "cardRadius": "24px"
  }
}

```

Если точные значения определить нельзя, использовать приблизительные.

10. Шаг 6 – генерация frontend-кода

Нужно сгенерировать минимальный frontend-проект.

Стек:

Next.js;
 React;
 TypeScript;
 Tailwind CSS.

Проект должен находиться здесь:

runs/run-001/code/frontend-project/

Требования:

только одна страница;
 только один hero-блок;
 структура должна быть простой;
 текст должен соответствовать брифу;
 дизайн должен повторять reference image;
 не использовать сторонние UI-библиотеки;
 не добавлять лишние секции;
 не менять композицию;
 не делать редизайн.

Минимальная структура:

```

frontend-project/
  app/
    page.tsx
    layout.tsx
    globals.css

  components/
    Hero.tsx

```

public/

package.json
tailwind.config.ts
tsconfig.json

11. Шаг 7 – проверка запуска проекта

После генерации кода нужно проверить:

устанавливаются ли зависимости;
запускается ли проект;
нет ли ошибок сборки;
нет ли отсутствующих файлов;
нет ли TypeScript-ошибок.

Если проект не запускается, нужно создать отчёт:

runs/run-001/qa/build-error.md

После этого нужно попробовать исправить только технические ошибки.

Важно:

не менять дизайн;
не менять текст;
не переписывать всё заново без необходимости;
исправлять только ошибки запуска/сборки.

Максимум попыток исправления сборки: 2.

12. Шаг 8 – создание скриншотов

Когда проект успешно запускается, нужно открыть его в браузере и сделать скриншоты.

Нужно сделать два скриншота:

runs/run-001/screenshots/rendered-desktop.png
runs/run-001/screenshots/rendered-mobile.png

Размеры:

desktop: 1440x900
mobile: 390x844

13. Шаг 9 – сравнение референса и результата

Нужно сравнить:

reference/hero-reference.png

c

screenshots/rendered-desktop.png

Проверить:

похожа ли композиция;
совпадает ли расположение заголовка;
похожи ли размеры текста;
совпадает ли цветовая схема;
похожи ли кнопки;
похожа ли карточка справа;
не потерялись ли элементы;
не появился ли лишний мусор;
похож ли общий визуальный стиль.

На выходе создать файл:

runs/run-001/qa/visual-report.md

Формат отчёта:

Visual QA Report

Общая оценка

Например: 7/10

Что совпало

- Цветовая схема близка.
- Композиция в целом похожа.
- Основные элементы на месте.

Что не совпало

- Заголовок слишком маленький.
- Правая карточка расположена ниже, чем в референсе.
- Фон недостаточно глубокий.
- Кнопка отличается по размеру.
- Нет нужного свечения вокруг карточки.

Критичные проблемы

- Нет / есть.

Рекомендации для исправления

1. Увеличить заголовок на 15–20%.
2. Поднять правую карточку выше.
3. Усилить фоновый градиент.
4. Увеличить высоту основной кнопки.
14. Шаг 10 – финальный статус

После всех шагов нужно обновить:

runs/run-001/status.json

Возможные статусы:

completed
reference_failed
build_failed
visual_failed
needs_manual_review

Для первой версии не нужно автоматически чинить визуальные отличия.

Достаточно:

сгенерировать визуал;
сгенерировать код;
сделать скриншот;
создать отчёт отличий.
15. Критерий готовности MVP

Минимальный прототип считается готовым, если:

Можно положить бриф в input/brief.md.
Система создаёт визуальный референс hero-блока.
Система создаёт описание дизайна.
Система создаёт Next.js-проект.
Проект запускается.
Система делает скриншот результата.
Система создаёт отчёт, насколько результат похож на референс.
16. Что не оптимизировать сейчас

Пока не нужно оптимизировать:

скорость;
стоимость генерации;
интерфейс;
красивую архитектуру;
авторизацию;
хранение пользователей;
оплату;
деплой;
идеальную мобильную версию;
поддержку всех типов сайтов.

Сейчас нужно доказать только одно:

можно ли стабильно превратить AI-generated visual reference в похожий frontend-код.

17. Следующий этап, но не делать сейчас

После завершения этого MVP можно будет добавить:

Автоматический repair loop.
Генерацию 3 блоков: Hero, Features, CTA.
ZIP-экспорт.
Простейший локальный UI.
Сравнение нескольких вариантов дизайна.
Историю запусков.
Early access SaaS.

Но это не входит в текущую задачу.